Create Abstract Classes for Shared Partial Behavior (Interview Notes)

What is an Abstract Class?

An **abstract class** is a class that **cannot be instantiated (cannot create objects)**. It is used when multiple related classes share some common behavior, while each subclass provides its own implementation for certain methods.

It is declared using the abstract keyword.

```
abstract class Animal {

    void eat() {
        System.out.println("Eating");
    }

    abstract void sound();
}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

Why Use an Abstract Class?

Use an abstract class when:

- Multiple classes have **common code**.
- Some methods should have a **default implementation**.
- Some methods **must be implemented** by every subclass.

This avoids code duplication and enforces a common structure.

Real-world Example

```
abstract class Employee {

    void login() {
        System.out.println("Employee Logged In");
    }

    abstract double calculateSalary();
}

class FullTimeEmployee extends Employee {

    @Override
    double calculateSalary() {
        return 50000;
    }
}

class PartTimeEmployee extends Employee {

    @Override
    double calculateSalary() {
        return 20000;
    }
}
```

Here,

- login() is common for all employees.
- calculateSalary() differs for each employee.

Important Interview Points

- Cannot create an object of an abstract class.
- Can contain **abstract methods** and **normal (concrete) methods**.
- Can have constructors, fields, static methods, and final methods.
- A subclass **must implement all abstract methods**, otherwise it must also be declared abstract.

Abstract Class vs Concrete Class

Abstract Class	Normal Class
Cannot create objects	Can create objects
Can have abstract methods	Cannot have abstract methods
Used as a base class	Used directly

Tricky Interview Questions

Q1. Can we create an object of an abstract class?

Answer: No.

```
Animal a = new Animal();   // ❌ Compile Error
```

Q2. Can an abstract class have constructors?

Answer: Yes. The constructor runs when a subclass object is created.

Q3. Can an abstract class have only concrete methods?

Answer: Yes.

```
abstract class Test {
    void show() {}
}
```

This is valid.

Q4. Can an abstract class contain variables?

Answer: Yes.

It can contain:

- Instance variables
- Static variables
- Final variables

Q5. Can an abstract method be private?

Answer: No.

Private methods cannot be overridden.

Q6. Can an abstract class be final?

Answer: No.

- abstract means **must be inherited**.
- final means **cannot be inherited**.

These keywords contradict each other.

Q7. What happens if a subclass doesn't implement all abstract methods?

Answer: The subclass must also be declared abstract.

Q8. Can an abstract class implement an interface?

Answer: Yes.

It may choose to implement the interface methods or leave them for its subclasses.

Interview Summary

- **Abstract class = Partial implementation + Common behavior**
- **Cannot instantiate directly**
- **Can have both abstract and concrete methods**
- **Used when related classes share code but require different implementations**
- **Supports constructors, fields, static methods, and final methods**